

Learning to Route

An RL-Gated Policy for Hierarchical Speculative Decoding

viasd-rl

June 20, 2026

One breath: speculative decoding speeds up LLMs; VIA-SD makes it smarter with a slim middle verifier — but it routes between tiers with *hand-picked thresholds*. We recast that routing as a learned decision problem and get a Pareto improvement over the hand-tuned baseline.

Overview

We introduce the first reinforcement-learned routing policy for hierarchical speculative decoding.

Speculative decoding accelerates LLM inference; VIA-SD adds a slim intermediate verifier but routes between tiers with *hand-tuned confidence thresholds*.

We present **three key innovations**:

1. a **learned per-token gating policy** that recasts tier-routing as a sequential decision problem — a tiny q -free MLP replaces the fixed thresholds;
2. a **quality–cost reward** that balances matching the target model against expensive-call latency;
3. a **per-token RL formulation** with principled credit assignment (REINFORCE $\rightarrow$ GRPO $\rightarrow$ per-token advantage).

The payoff: we match *lossless* accuracy (**0.91**) at **4 $\times$ fewer** full-model calls (**2.2 $\times$ faster** than greedy), **Pareto-dominate** the paper's VIA-SD (**$\sim 2\times$ its speedup at 2.3 $\times$ its accuracy**), and reach up to **3.5 $\times$** when pushed for speed.

Learning to Route

└ Overview

This is the abstract. Strong claim first: the first *learned* router for hierarchical speculative decoding. Then the gap: VIA-SD's tiers are routed by hand-tuned thresholds. Then our three contributions — the learned gate, the reward that trades quality against cost, and the RL training with per-token credit assignment. And the thesis we challenge: that you need to hand-tune thresholds at all. Spend the rest of the talk unpacking these three.

Overview

We introduce the first reinforcement-learned routing policy for hierarchical speculative decoding.

Speculative decoding accelerates LLM inference; VIA-SD adds a slim intermediate verifier but routes between tiers with hand-tuned confidence thresholds.

We present **three key innovations**:

1. a **learned per-token gating policy** that recasts tier-routing as a sequential decision problem — a tiny q-free MLP replaces the fixed thresholds;
2. a **quality-cost reward** that balances matching the target model against expensive-call latency;
3. a **per-token RL formulation** with principled credit assignment (REINFORCE → GRPO → per-token advantage).

The payoff: we match *lossless* accuracy (0.91) at 4× **fewer** full-model calls (2.2× faster than greedy). **Pareto-dominates** the paper's VIA-SD (~2× its **speedup** at 2.3× its accuracy), and reach up to 3.5× when pushed for speed.

The bottleneck: LLM decoding is slow and sequential

- Each token **streams the entire model** from memory — inference is *memory-bandwidth bound*.
- **Speculative decoding (SD)**: a cheap *drafter* guesses several tokens; the big *verifier* checks them in **one parallel pass**.
- Accepted guesses $\Rightarrow$ many tokens per expensive forward. **Lossless**, $\sim 1.4\times$ wall-clock.
- The one lever: the **acceptance rate** — every rejected token forces a full-model step.

└ The bottleneck: LLM decoding is slow and sequential

- Each token **streams the entire model** from memory — inference is *memory-bandwidth bound*.
- **Speculative decoding (SD)**: a cheap *drafter* guesses several tokens; the big *verifier* checks them in **one parallel pass**.
- Accepted guesses → many tokens per expensive forward. **Lossless**, ~1.4x wall-clock.
- The one lever: the **acceptance rate** — every rejected token forces a full-model step.

Frame the cost. Generating each token reads every weight from memory — the sequential bottleneck behind latency and serving cost. SD's trick: a small drafter proposes the next few tokens, and the big verifier checks them all in one parallel forward. Accepted tokens ride free; the output is identical to normal decoding. 1.4x wall-clock on our pair. The whole game is the acceptance rate.

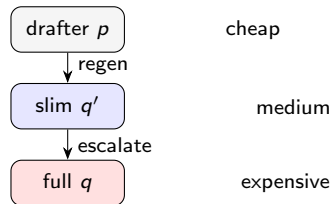
VIA-SD improves on vanilla SD

Vanilla SD — binary per token:

- ACCEPT drafter, or REJECT $\rightarrow$ full model q .

VIA-SD — adds a **slim middle verifier** q' (q with $\sim 45\%$ of layers skipped):

- ACCEPT — REGENERATE (q') — ESCALATE (q).
- “Middle-zone” tokens handled *cheaply* by $q' \Rightarrow$ fewer full-model calls.



Learning to Route

└ VIA-SD improves on vanilla SD

Vanilla SD — binary per token:

- ACCEPT *drafter*, or REJECT → full model *q*.

VIA-SD — adds a **slim middle verifier** q' (q with ~45% of layers skipped):

- ACCEPT ⇒ REGENERATE (q') ⇒ ESCALATE (q).
- “Middle-zone” tokens handled cheaply by q' ⇒ fewer full-model calls.



Vanilla SD is all-or-nothing. VIA-SD’s insight: many “rejected” tokens are borderline. So it adds a cheaper middle judge — the same big model with 45% of layers skipped — and makes a three-way choice: accept the drafter, regenerate with q' , or escalate to the full model. Borderline tokens go to the cheap tier, so you call the full model far less. But how does it *decide*?
Next slide.

...but it routes with hand-picked thresholds

VIA-SD's routing rule

Two fixed confidence thresholds on q' : $\alpha_1 = 0.5$ $\alpha_2 = 0.3$

- **Global & static** — one cutoff for every token and context.
- **Brittle** — must be re-tuned per model and task.
- **Sub-optimal** — even fully swept, accuracy peaks at ≤ 0.50 on our setup.

*Choosing a tier per token is a sequential, context-dependent decision — so **learn** it.*

Learning to Route

└ ... but it routes with *hand-picked thresholds*

The opening. VIA-SD's three-way choice is two hand-picked numbers, 0.5 and 0.3 — global, static, context-blind, and re-tuned per model/task. Even sweeping all nine configs, accuracy caps at 0.50 on our setup. But picking a tier per token from context is exactly a sequential decision problem. So learn the policy. That sets up our three innovations.

VIA-SD's routing rule

Two fixed confidence thresholds on q^i : $\tau_1 = 0.5$ $\tau_2 = 0.3$

- **Global & static** — one cutoff for every token and context.
- **Brittle** — must be re-tuned per model and task.
- **Sub-optimal** — even fully swept, accuracy peaks at ≤ 0.50 on our setup.

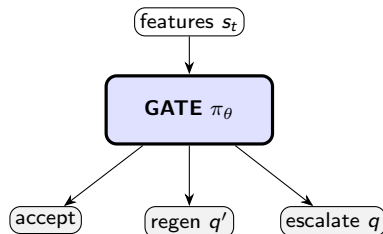
Choosing a tier per token is a sequential, context-dependent decision — so **learn it**.

Innovation 1/3 — a learned per-token gating policy (MDP)

Recast tier-routing as a **Markov decision process**:

- **State** s_t : q -free features — drafter/ q' confidence, agreement, $\text{KL}(p\|q')$, position.
- **Action** $a_t \in \{\text{accept, regenerate, escalate}\}$.
- **Policy** $\pi_\theta(a_t | s_t)$: a tiny MLP.

Key property: q is consulted only to *score* at train time — **never** a gate input, so the gate is free to run at inference.



Learning to Route

Method

Innovation 1/3 — a learned per-token gating policy (MDP)

Innovation one: the learned gate. We formalize routing as an MDP. The state is a handful of cheap features — drafter and q-prime confidence, whether they agree, their KL, position in the block. The action is the tier. The policy is a tiny MLP. The crucial design choice: the features are q-free, so the full model is used only to compute rewards during training; at inference the gate decides without ever running q. That makes it free to deploy — no extra serving model.

Recast tier-routing as a **Markov decision process**:

- **State** s_t : q-free features — drafter/q' confidence, agreement, $KL(p||q')$, position.
- **Action** $a_t \in \{\text{accept, regenerate, escalate}\}$.
- **Policy** $\pi_t(a_t | s_t)$: a tiny MLP.

Key property: q is consulted only to score at train time — never a gate input, so the gate is free to run at inference.



Innovation 2/3 — a quality–cost reward

Two ingredients per token:

$$\text{quality: } \text{match}_t = \mathbf{1}[\hat{y}_t = \arg \max_v q(v \mid x_{<t})]$$

$$\text{cost: } \text{cost}_t = (t_{p_1} + t_{q'}/\gamma + \mathbf{1}[\text{ESCALATE}] t_q) / t_{q_1}$$

Reward (cost in units of one full-model step):

$$\text{trajectory: } R = \overline{\text{match}} - \lambda \overline{\text{cost}} \text{ (+ } r_c \text{ correct)} \qquad \text{per-token: } r_t = \text{match}_t - \lambda \text{cost}_t$$

λ is the **quality↔speed knob**; `match` is a *dense* proxy for the sparse true objective (final-answer correctness).

Learning to Route

└ Method

└ Innovation 2/3 — a quality–cost reward

Two ingredients per token:

$$\text{quality: } \text{match}_t = \mathbb{1}[\hat{y}_t = \arg \max_v q(v \mid x_{<t})]$$

$$\text{cost: } \text{cost}_t = (t_{p_t} + t_{q_t} / \gamma + \mathbb{1}[\text{ESCALATE}] t_{q_t}) / t_{q_0}$$

Reward (cost in units of one full-model step):

$$\text{trajectory: } R = \overline{\text{match}} - \lambda \overline{\text{cost}} \quad (+ r_t \text{ correct}) \quad \text{per-token: } r_t = \text{match}_t - \lambda \text{cost}_t$$

λ is the **quality–speed knob**; match is a dense proxy for the sparse true objective (final-answer correctness).

Innovation two: the reward. Quality is whether the emitted token matches what the full model would greedily produce — computed by running q at train time only. Cost is the per-action latency: drafter step, plus the slim q -prime amortized over the block, plus the full model only if we escalate, all normalized by one full-model step. The reward subtracts λ times cost from match; λ is the quality-versus-speed knob. We use both a trajectory form and a per-token form. One caveat we measured: match is a dense proxy — 98.5% token match still leaves 0.72 final accuracy, because a few token slips derail the chain.

Innovation 3/3 — per-token RL with credit assignment

Policy gradient (all variants, $+\eta \mathcal{H}[\pi_\theta]$ entropy):

$$\nabla_\theta J = \mathbb{E} \left[\sum_t A_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right]$$

They differ in the **advantage** A_t :

- **REINFORCE** (trajectory): $A = R - b$, EMA baseline $b \leftarrow (1-\tau)b + \tau R$.
- **GRPO** (group): $A_i = \frac{R_i - \text{mean}_G R}{\text{std}_G R}$, with PPO-clip $\rho_t = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$ and KL to π_{ref} .
- **per-token (v2)**: $\hat{A}_t = \frac{r_t - \text{mean}_{\text{batch}} r}{\text{std}_{\text{batch}} r}$ — local credit, **lowest variance**.

Learning to Route

└ Method

└ Innovation 3/3 — per-token RL with credit assignment

Innovation three: how we optimize. Same policy-gradient skeleton — advantage times grad-log-prob, plus an entropy bonus. The methods differ only in the advantage. REINFORCE broadcasts one trajectory return minus an EMA baseline over all decisions — crude and high-variance. GRPO compares rollouts of the same prompt, normalizing by the group, with PPO clipping and a KL leash to the warm-start — better for sparse rewards. Our per-token v2 gives each decision its own batch-normalized advantage — since routing effects are local, this is the lowest-variance and works best. Credit assignment is the key axis.

Policy gradient (all variants, + $\eta H[\pi_\theta]$ entropy):

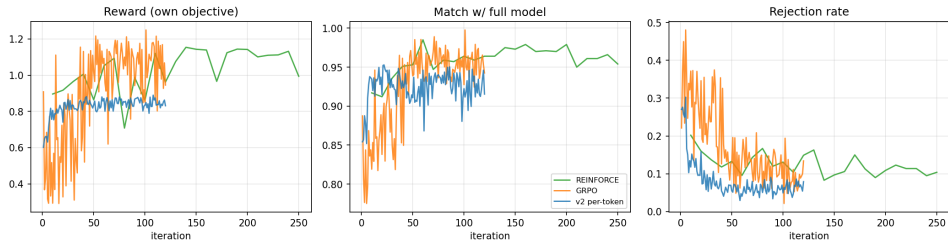
$$\nabla_{\theta} J = \mathbb{E} \left[\sum_t A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

They differ in the **advantage** A_t :

- **REINFORCE** (trajectory): $A = R - \hat{b}$, EMA baseline $\hat{b} \leftarrow (1-\tau)\hat{b} + \tau R$.
- **GRPO** (group): $A_t = \frac{R_t - \text{mean}_g R}{\text{std}_g R}$, with PPO-clip $\rho_t = \frac{\pi_{\theta_t}}{\pi_{\text{ref}}}$ and KL to π_{ref} .
- **per-token (v2)**: $\hat{A}_t = \frac{r_t - \text{mean}_{\text{batch}} r}{\text{std}_{\text{batch}} r}$ — local credit, **lowest variance**.

Training converges — per-token (v2) is cleanest

RL training curves (0.5B→14B, GSM8K)



Learning to Route

Results

└ Training converges — per-token (v2) is cleanest

All three converge: reward up, token-match 0.95, rejection down. The difference is variance — GRPO (orange) noisy, REINFORCE (green) middling, per-token v2 (blue) smoothest and lowest rejection. Evidence for the credit-assignment point on the previous slide.



Results: the learned gate dominates

Method	GSM8K acc	q-calls/tok	speedup	
greedy- q (reference)	0.887	1.000	1.00×	
plain SD (standard, lossless)	0.907	0.259	1.40×	[†]
via_fixed (paper, tuned)	0.40	0.31	1.76×	[†] wall-clock; via_*
via_imit (imitation only)	0.34	0.29	1.69×	
via_rl REINFORCE	0.880	0.243	—	
via_rl GRPO	0.853	0.162	—	
via_rl REINFORCE+DIMR	0.907	0.250	2.22×	
via_rl v2 per-token	0.713	0.104	3.53×	

speedups are the overhead-free bandwidth model (not directly comparable).

Learning to Route

Results

Results: the learned gate dominates

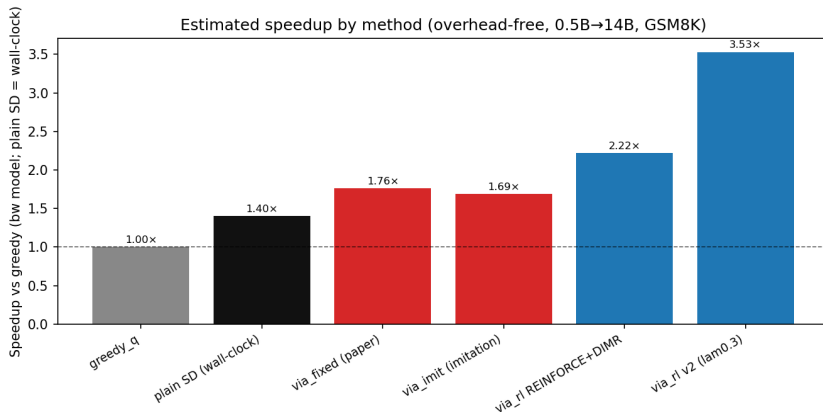
Results: the learned gate dominates

Method	GSMK acc	q-calls/tok	speedup
greedy-q (reference)	0.887	1.000	1.00x
plain SD (standard, lowlat)	0.907	0.259	1.40x†
via_fixed (paper, tuned)	0.40	0.31	1.76x
via_min (imitation only)	0.34	0.20	1.69x
via† REINFORCE	0.880	0.243	—
via† GRPO	0.853	0.162	—
via† REINFORCE+DIMR	0.907	0.250	2.22x
via† v2 per-token	0.713	0.104	3.53x

speedups are the overhead-free bandwidth model (not directly comparable).

Full table. Reference at top (0.89–0.91). Red = fixed thresholds: 0.34–0.40, broken. Blue = learned gates: REINFORCE and REINFORCE+DIMR recover full accuracy; GRPO drives expensive calls lowest (0.162); v2 trades accuracy for top speed. Honesty note: plain SD's 1.4x is wall-clock, via_* are an idealized bandwidth model — compare on q-calls/token, where the learned gates clearly win.

Latency: speedup by method



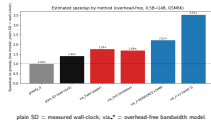
plain SD = measured wall-clock; via_* = overhead-free bandwidth model.

Learning to Route

Results

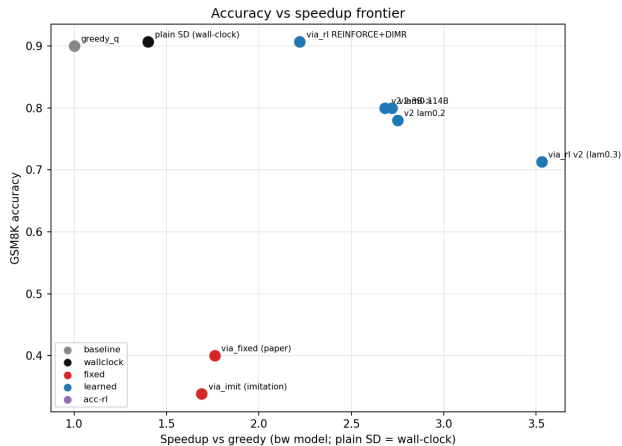
Latency: speedup by method

Latency: speedup by method



Greedy is the 1x baseline. The fixed-threshold gate (red) is slow — it over-escalates. Learned gates (blue) push speedup up; per-token v2 tops the chart by keeping escalation below break-even. Absolute heights mix metrics (wall-clock vs bandwidth) — the message is the ordering.

The accuracy–speed frontier



Learning to Route

Results

The accuracy-speed frontier



The design space. Red = fixed-threshold, bottom-left, dominated. Blue = learned gates, up and to the right. The lambda knob exposes a real frontier — accuracy (0.80) or speed (q-calls/token 0.10). And a bigger *drafter* (3B) hits 0.80 with 6x fewer verifier calls — the right lever. The learned policy Pareto-dominates the fixed baseline.

Impact: a new accuracy–efficiency frontier

- **Lossless quality, 4× cheaper.** Match greedy/lossless accuracy (**0.91**) at 4× **fewer** full-model calls — a 2.2× speedup, *zero accuracy lost*.
- **A tunable frontier.** One knob: 2.7× faster at −0.1 accuracy, or 3.5× at −0.19 — up to 10× **fewer** full-model calls.
- **Pareto-dominates VIA-SD.** $\sim 2\times$ its speedup *and* +0.3 **to** +0.5 accuracy; where its fixed thresholds collapse to 0.40, our gate holds **0.71–0.91**.
- **Scales with the drafter.** A 3B drafter $\rightarrow$ 36× **fewer** full-model calls at 0.80 accuracy — and it's a free-to-deploy tiny MLP (q -free, no q at decision time).

Stop hand-picking thresholds — let the model schedule its own compute.

$\times$ = bandwidth model; “fewer calls” (q -calls/token) is exact & hardware-independent.

Learning to Route

└ Results

└ Impact: a new accuracy–efficiency frontier

Hit the numbers hard. One: we match lossless accuracy — 0.91 — at four times fewer full-model calls than greedy, a 2.2x speedup, with zero accuracy lost. Two: it's a tunable frontier — 2.7x faster for a tenth of a point of accuracy, or 3.5x for about 0.19, up to ten times fewer expensive calls. Three: we Pareto-dominate the paper's VIA-SD — roughly twice its speedup AND 0.3 to 0.5 higher accuracy; where its fixed thresholds collapse to 0.40, we hold 0.71 to 0.91. Four: it scales with the drafter — a 3B drafter gives 36x fewer full-model calls at 0.80 accuracy — and it's a free-to-deploy tiny MLP that never touches the big model to decide. Tagline: stop hand-picking thresholds; let the model schedule its own compute.

- **Lossless quality, 4x cheaper.** Match greedy/lossless accuracy (0.91) at 4x fewer full-model calls — a 2.2x speedup, zero accuracy lost.
- **A tunable frontier.** One knob: 2.7x faster at ~0.1 accuracy, or 3.5x at ~0.19 — up to 10x fewer full-model calls.
- **Pareto-dominates VIA-SD.** ~2x its speedup and +0.3 to +0.5 accuracy; where its fixed thresholds collapse to 0.40, our gate holds **0.71–0.91**.
- **Scales with the drafter.** A 3B drafter → 36x fewer full-model calls at 0.80 accuracy — and it's a free-to-deploy tiny MLP (q-free, no q at decision time).

Stop hand-picking thresholds — let the model schedule its own compute.

x = bandwidth model, "fewer calls" (q-calls/token) is exact & hardware-independent

Future work

- **Spec-spec decoding:** make the drafter *itself* speculative — a recursive hierarchy; the gate routes across *levels*.
- **KnapSpec:** verification as a **knapsack** — a fixed budget of expensive calls spent on the highest-value tokens (value = correctness impact, weight = cost).
- **Correctness-floor RL:** minimize cost *subject to* an accuracy floor (avoids reward-hacking collapse).
- **Scale:** bigger drafters, more tasks, real-engine wall-clock (static cache / CUDA graphs).

Learning to Route

└ Results

└ Future work

- **Spec-spec decoding:** make the drafter itself speculative — a recursive hierarchy; the gate routes across levels.
- **KnapSpec:** verification as a **knapsack** — a fixed budget of expensive calls spent on the highest-value tokens (value = correctness impact, weight = cost).
- **Correctness-floor RL:** minimize cost subject to an accuracy floor (avoids reward-hacking collapse).
- **Scale:** bigger drafters, more tasks, real-engine wall-clock (static cache / CUDA graphs).

Where it goes. Spec-spec: the drafter is itself slow, so make it speculative too — a recursive tree, gate routing across levels. KnapSpec: spend a fixed budget of expensive calls like a knapsack, on the tokens that most change the answer — directly targeting the match-vs-accuracy gap. Correctness-floor RL: minimize cost subject to an accuracy floor, dodging the collapse from naive correctness rewards. And scale it — bigger drafters, more tasks, real wall-clock on an optimized engine.

Learning to Route

Stop hand-picking thresholds — let the model schedule its own compute.

Thanks! questions?